

课程笔记

MIT RES.9-009 第 02 讲: Neuroblox 图形界面

MIT RES.9-009 课程整理 & Codex

2026 年 4 月 15 日



视频作者/频道: MIT Video Productions; 讲者 Helmut H. Strey

发布日期: 2025-04-10 (讲课日期: 2025-01-13)

视频时长: 20:51

视频链接: <https://www.youtube.com/watch?v=X1BRJps84zE>

目录

1	GUI 入口、flat brain 与协作状态	2
1.1	本章小结	3
2	Harmonic Oscillator: 最小可运行的双节点系统	3
2.1	本章小结	4
3	Cortical demo: 从单块到双区域	5
3.1	本章小结	6
4	总结与延伸	6
4.1	本章小结	6

证据层说明

本讲没有找到独立的官方 slide PDF。正文主要依据字幕、官方课程页和视频帧；课程页中的 logo 首图只承担品牌展示，不承担本讲教学内容，因此已在 `omission_log.jsonl` 中明确记录。

1 GUI 入口、flat brain 与协作状态

这一讲一开始就把两件事说得很明确。第一，演示的是 Neuroblox 的新 GUI，而不是 JuliaHub 上那个完全稳定的版本，因为还有一些功能没有完全接通。第二，Neuroblox 现在已经进入可以给学生试用的阶段，但它仍然是一个快速迭代的系统，所以讲者反复提醒，遇到 bug 不要惊讶，要把它当成真实协作的一部分。

官方课程页在这个层面上也给了同样的背景：Neuroblox 被定位成一个基于 Julia 的模块化平台，支持从单个 block 到更大系统的拼接，并允许通过 graphical interface 直接组装模型。第 2 讲的作用，就是把这个抽象定位落到界面上，告诉你到底怎么点、怎么连、怎么跑。



图 1: GUI 总览: flat brain、节点区和右侧属性面板同时可见。这个画面对应讲者解释 canvas、sidebar 和 simulation 面板的段落（约 00:08:05）。

最核心的交互语义其实很简单，但必须记牢。canvas 上的脑区是一个 flat map，也就是把 brain region 展开到平面上；把 block 放到某个区域上以后，你就可以通过 Alt 或 Option 建边。左键点击是选择，左键拖拽边上的 handle 可以改变边的几何形状。换句话说，GUI 里既有“结构放置”，也有“连接编辑”，这两者不是同一个动作。

几个必须记住的操作语义

Alt 或 Option 用来建边；左键点 node 是选 node；左键点 edge 是选 edge；拖拽 edge anchor 可以改边形状。这个交互设计的目的，是让你在不离开 flat brain 的情况下同时编辑结构和动力学。

右侧面板的结构也很明确。Nodes & Edges 用来改节点和连接的属性，Params 用来集中管理参数，Sim 负责仿真设置，Parameter Fitting 和 Reinforcement Learning 目前都还不够稳定，Data Loader 则负责加载数据。讲者特地提醒，不要把还没完全工作的功能当成课程主流程的一部分。

当前可用性提醒

讲者明确说过，parameter fitting 目前不建议依赖，reinforcement learning 也还没有在这个 GUI 上形成稳定经验。教学上应把它们当成未来功能，而不是当前练习的主线。

另一个很现实的细节是云端启动时间。连接 JuliaHub 后，系统要在 AWS 里预留计算资源，随后 Julia 又要进行 JIT 编译，所以第一次点开和第一次跑仿真会慢很多。讲者把这一点解释得很直白：不是 GUI “坏了”，而是云服务和编译开销在前面排队。

你还可以把这一讲里的平台架构理解成三层协作：前端负责界面，graphics server 负责作图，Julia backend server 负责真正的计算。讲者说得很清楚，这三者是互相通信的；等仿真结束，数据先传给图形服务器，再回到界面里显示。这也是为什么 GUI 看起来是“点一下就出图”，但背后其实是一条很长的异步链路。

1.1 本章小结

这一部分不是在讲“GUI 好不好看”，而是在讲 Neuroblox 的操作语义：flat brain 负责空间布局，Alt/Option 负责连边，右侧面板负责参数与仿真，云端和 Julia 则负责执行。把这些语义记牢，后面的示例才有操作基础。

2 Harmonic Oscillator：最小可运行的双节点系统

讲者接下来选了一个最小但很适合练手的例子：harmonic oscillator neural mass model。它足够简单，能让你快速摸到 GUI 的所有关键操作；它也足够接近动力系统的本体，让你理解为什么 Neuroblox 不是“画图工具”，而是“搭动力学系统的工具”。

最基本的连续形式可以写成

$$\dot{x}(t) = y(t), \quad \dot{y}(t) = -\omega^2 x(t) - \gamma y(t) + I_{\text{couple}}(t).$$

如果只看无阻尼版本，那么把一阶系统消元后就是

$$\ddot{x}(t) + \omega^2 x(t) = 0,$$

解是正弦或余弦，频率由 ω 控制。讲者进一步提醒，若只有阻尼项而没有合适的耦合，这个系统会衰减掉，不会形成你想要的稳定振荡。所以在 GUI 里，他不是只放一个 oscillator，而是放两个互相作用的 oscillator。

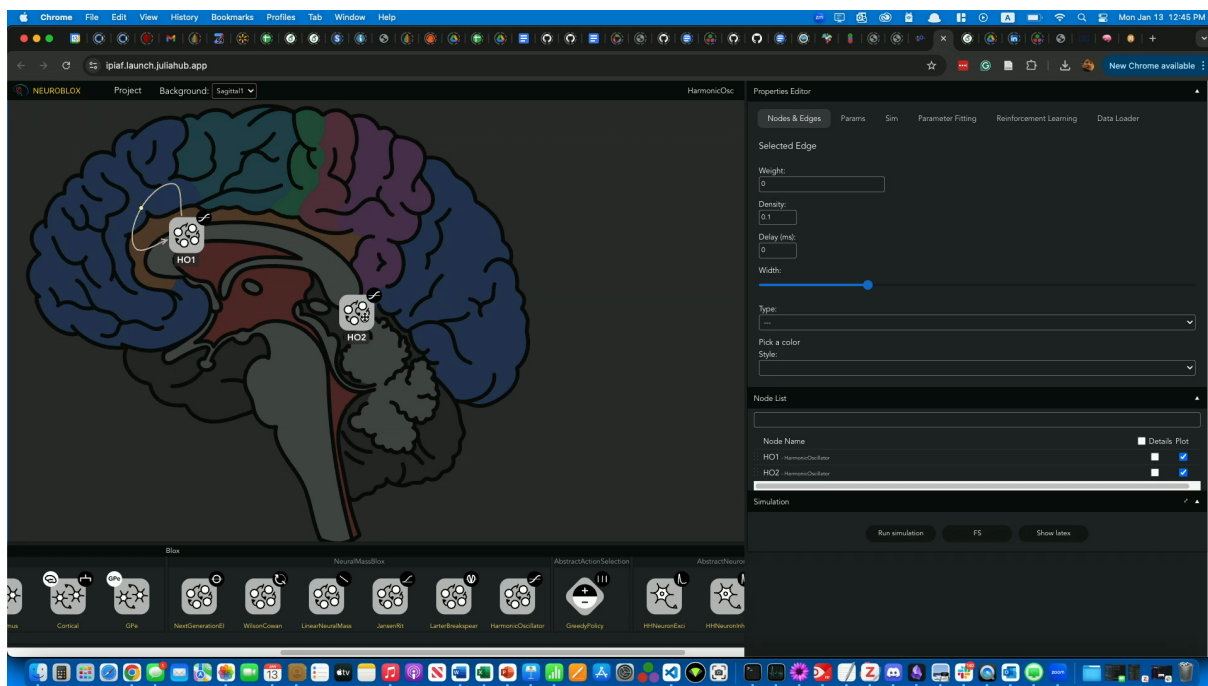


图 2: 两个 harmonic oscillator, HO1 和 HO2 已经放到 canvas 上, 并且能在右侧看到边的属性面板。这个状态对应讲者说明 self-interaction、Alt 拖拽和命名的片段 (约 00:10:02)。

讲者给出的实操顺序很稳定：先在底部 block 面板里找到 `HarmonicOsc`, 拖到 canvas 上, 重命名为 HO1 和 HO2, 然后给每个 node 加 self-interaction, 再通过 Alt/Option 从一个 node 连到另一个 node。起始参数并不是理论定值, 而是一个经验起点。讲者建议先把 self-interaction 设成 inhibitory, 也就是大约 -1, 再把跨节点连接设成更强一些的正值, 例如 5 和 6, 然后观察是否能把系统推到振荡区间。

为什么要用两个 oscillator

单个 damped oscillator 更像一个会衰减的振动器；两个相互作用的 oscillator 才更像一个可以通过耦合维持节律的系统。讲者在这里的重点不是某个固定参数, 而是让你感受“自耦合 + 互耦合”如何改变系统的相图。

GUI 里还给了一个很有用的事实：Neuroblox 的时间单位是 milliseconds, 所以输入的 1 不是秒, 而是 1 ms。仿真面板还允许你设定 duration、relative tolerance、absolute tolerance, 以及 stiff / non-stiff solver。讲者的建议是, 先让系统跑通, 再去调这些数值细节；第一次运行还会慢一些, 因为 Julia 需要先编译相关包, 而云端环境里这一步往往最耗时间。

当仿真跑完以后, 右侧的结果面板会显示轨迹, 你可以把图放大、挪到另一侧、切换查看 node 的参数, 甚至打开 `Show latex` 看系统方程。讲者还顺手提到, 一个想当然的小问题其实很适合作为练习：如果你已经把这个系统调到会振荡, 如何把它的频率加倍？这个问题没有在现场给出唯一答案, 它本来就是为了让你理解参数、耦合和频率之间的关系。

2.1 本章小结

这一部分的价值在于把 GUI 操作和动力系统直觉绑在一起。你不是在“摆两个图标”, 而是在构造一个能通过 self-interaction 和 cross-coupling 进入振荡区间的最小系统。会搭这个例子, 后面

的 cortical demo 才不是黑箱。

3 Cortical demo: 从单块到双区域

第三段把系统规模往上推了一层。讲者先新建一个 `cortical demo` 项目，然后从 `composite blocks` 里选出 `Cortical block`，把它放到 `canvas` 上并命名为 `C1`。这个例子说明了一件很重要的事：Neuroblox 的核心不是单个 `neuron`，而是 `blocks of blocks`。你可以把 `WTA`、`cortical region`、`stimulus block` 继续封装，再把这些封装好的块放进更大的系统。

在参数层面，`Cortical block` 会出现若干控制项。最常见的有 `N_wta`、`N_exci`、`G_syn_exci`、`G_syn_inhib` 和 `G_syn_ff_inhib`。讲者的实用建议很直接：如果默认的 `N_wta = 10` 太慢，就先把它减到 5，因为你当前的目标是理解 workflow，不是让一次演示把时间烧完。

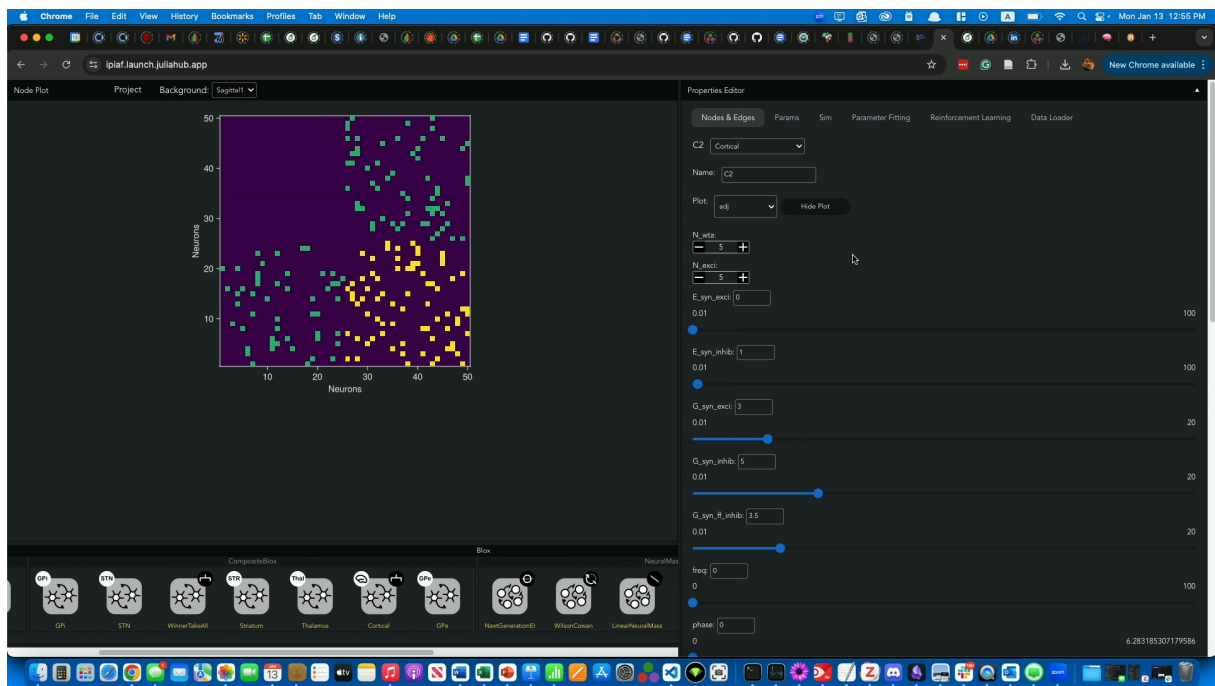


图 3: 双 cortical region 的结果视图：左侧显示 adjacency matrix / cross correlation 类输出，右侧是 C2 的参数面板。这个画面对应 00:19:22 左右的演示。

仿真完成后，GUI 提供的不只是一个时间序列，还包括多个特殊视图：`raster plot`、`stack plot`、`adjacency matrix`、`power spectrum`。讲者强调，`adjacency matrix` 里看到的连接是按 `density` 随机采样出来的，`density` 控制的是连接概率，不是突触权重；权重仍然要另行设置。这个区别非常重要，因为它直接决定你是在调结构稀疏性，还是调动力学强度。

density 不是 weight

`density` 决定“有多少条边会被采样出来”，`weight` 决定“已经采样出来的边有多强”。这两个概念必须分开，不然你会把结构问题和动力学问题混成一件事。

讲者随后把系统扩展成两个 cortical regions, `C1` 和 `C2`。在这个过程中，讲者又一次强调了 GUI 的实用细节：如果结果图还停留在旧参数上，就再点一次 `Run simulation`；如果 region 太大，仿真会明显变慢；如果想看更深的结构，还可以打开 `Show latex` 查看方程。更进一步，如果你愿意，

Neuroblox 还可以把 image stimulus block 接到 visual cortex，再把信号送到 prefrontal cortex。讲者把这部分留给后面愿意继续试的人，而不是把它当成今天必须完成的任务。

3.1 本章小结

这一段把 GUI 从“单个 oscillator 的练习场”推进成了“可拼装的 cortical system”。Cortical block、C1/C2、adjacency matrix 和 power spectrum 共同说明：Neuroblox 的可视化不是装饰，而是模型结构本身的一部分。

4 总结与延伸

这一讲真正教你的不是某个按钮，而是一整套操作顺序。先在 flat brain 上放 block，再用 Alt/Option 建边；先用 harmonic oscillator 验证你是否理解 node / edge / parameter 的关系，再把同样的逻辑推广到 cortical region 和 composite blocks。也就是说，GUI 不是课外附带工具，而是 Neuroblox 的建模入口。

如果把这讲压成三句话，就是：

- flat brain 让你把脑区、block 和连接放到同一个平面上编辑。
- HarmonicOsc 让你验证自己真的会建边、调参、跑仿真。
- Cortical block 让你看到单个 block 如何被组合成更大的回路系统。

这也是为什么讲者最后反复鼓励大家“多玩玩 GUI”。对后面的教程来说，能不能顺利进入 WinnerTakeAll、Cortical、ImageStimulus 等示例，取决于你是否已经把这一讲里的交互语义掌握住了。

课程位置

官方课程总览里，Blox and Connections in Neuroblox 之后紧接着就是 Neurons, Neural Masses and Sources、Circuit Models、Decision Making 等单元。本讲的作用，正是把你从“知道有这些单元”推进到“知道怎么真的在 GUI 里搭起来”。

4.1 本章小结

第 2 讲是后续所有教程的操作底座。它把界面、参数、仿真和复合块串成了一条完整的手工流程；只要这条链路打通，后面的更大回路和更复杂实验就只是规模上的扩展，而不是概念上的断层。